香港科技大學(廣州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

机器学习基础

讲师：Hao Wang,
AI Thrust，信息枢纽

鸣谢：大部分幻灯片修改自 Standford 的 CS231n

香港科技大學(廣州)
THE HONG KONG
UNIVERSITY OF SCIENCE AND
TECHNOLOGY (GUANGZHOU)

Machine Learning Basis

Instructor: Hao Wang
AI Thrust, Information Hub

Credit: A large portion of the slides are modified from CS231n in Standford

今日大纲

- 线性代数
- K 最近邻
- 支持向量机
- 梯度下降
- 计算图



Today's Outline

- Linear Algebra
- K Nearest Neighbor
- Support Vector Machine
- Gradient descent
- Computation graph



标量

- **简单操作**

$$c = a + b$$
$$c = a \cdot b$$
$$c = \sin a$$

- **长度**

$$|a| = \begin{cases} a & \text{if} a > 0 \\ -a & \text{otherwise} \end{cases}$$
$$|a + b| \leq |a| + |b|$$
$$|a \cdot b| = |a| \cdot |b|$$

Scalars

- **Simple operations**

$$c = a + b$$
$$c = a \cdot b$$
$$c = \sin a$$

- **Length**

$$|a| = \begin{cases} a & \text{if} a > 0 \\ -a & \text{otherwise} \end{cases}$$
$$|a + b| \leq |a| + |b|$$
$$|a \cdot b| = |a| \cdot |b|$$

向量

- **简单运算**

逐元素操作

$\boldsymbol{c} = \boldsymbol{a} + \boldsymbol{b}$ 其中 $c_i = a_i + b_i$

$\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b}$ 其中 $c_i = a_i b_i \boldsymbol{c} = \sin \boldsymbol{a}$

其中 $c_i = \sin a_i$

- **长度（范数）**

$$||\boldsymbol{x}|| = \sqrt{\sum_i x_i^2}$$

$$||\boldsymbol{a} + \boldsymbol{b}|| \leq ||\boldsymbol{a}|| + ||\boldsymbol{b}||$$

$$||a \cdot \boldsymbol{b}|| = |a| \cdot ||\boldsymbol{b}||$$

Vectors

- **Simple operations**

Element-wise operation

$\boldsymbol{c} = \boldsymbol{a} + \boldsymbol{b}$ where $c_i = a_i + b_i$

$\boldsymbol{c} = \boldsymbol{a} \cdot \boldsymbol{b}$ where $c_i = a_i b_i$

$\boldsymbol{c} = \sin \boldsymbol{a}$ where $c_i = \sin(a_i)$

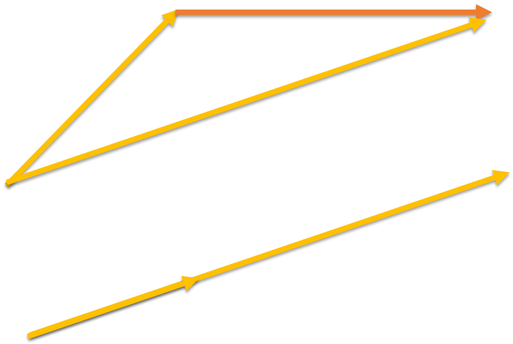
- **Length (Norm)**

$$||\boldsymbol{x}|| = \sqrt{\sum_i x_i^2}$$

$$||\boldsymbol{a} + \boldsymbol{b}|| \leq ||\boldsymbol{a}|| + ||\boldsymbol{b}||$$

$$||a \cdot \boldsymbol{b}|| = |a| \cdot ||\boldsymbol{b}||$$

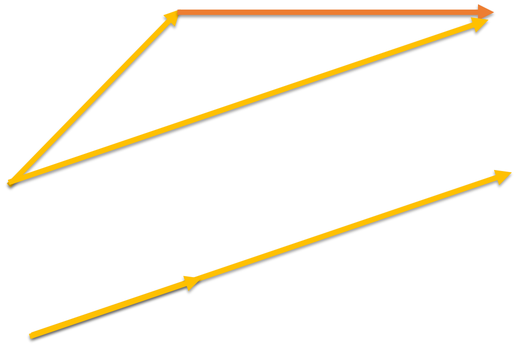
向量



$c = a + b$

$c = \alpha \cdot b$

Vectors



$c = a + b$

$c = \alpha \cdot b$

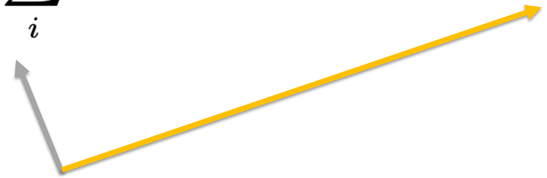
向量

- **点积**

$$a^\top b = \sum_i a_i b_i$$

- **正交：点积 □□**

$$a^\top b = \sum_i a_i b_i = 0$$



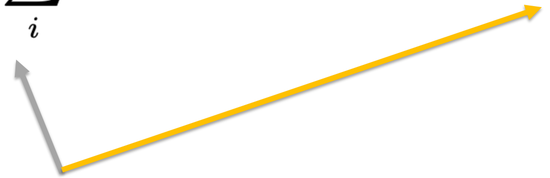
Vectors

- **Dot product**

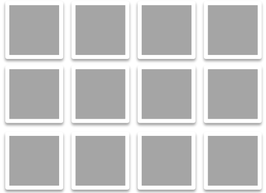
$$a^\top b = \sum_i a_i b_i$$

- **Orthogonal: dot product = 0**

$$a^\top b = \sum_i a_i b_i = 0$$



矩阵

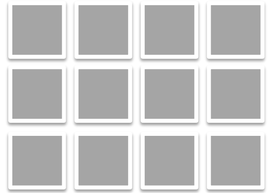


- **简单运算**

$$C = A + B \qquad \text{where } C_{ij} = A_{ij} + B_{ij}$$
$$C = \alpha \cdot B \qquad \text{where } C_{ij} = \alpha B_{ij}$$
$$C = \sin A \qquad \text{where } C_{ij} = \sin A_{ij}$$



Matrices



- **Simple operations**

$$C = A + B \qquad \text{where } C_{ij} = A_{ij} + B_{ij}$$
$$C = \alpha \cdot B \qquad \text{where } C_{ij} = \alpha B_{ij}$$
$$C = \sin A \qquad \text{where } C_{ij} = \sin A_{ij}$$



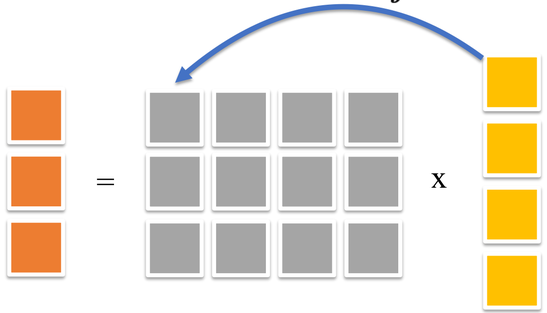
矩阵

- **乘法（矩阵向量）**

$$c = Ab \text{ where } c_i = \sum_j A_{ij} b_j$$

形状：Axb 矩阵乘法 b x cà A x C

Axbxc 矩阵乘法 cxd ->Axbxd



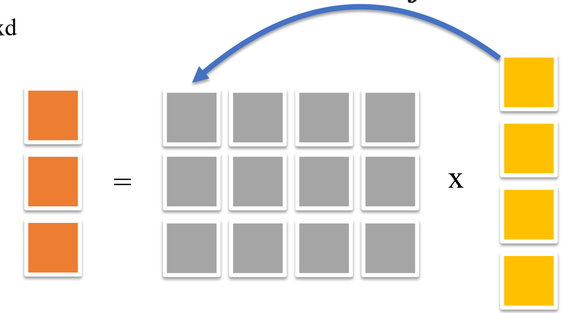
Matrices

- **Multiplications (matrix vector)**

$$c = Ab \text{ where } c_i = \sum_j A_{ij} b_j$$

Shape: Axb matmul b x c→ A x C

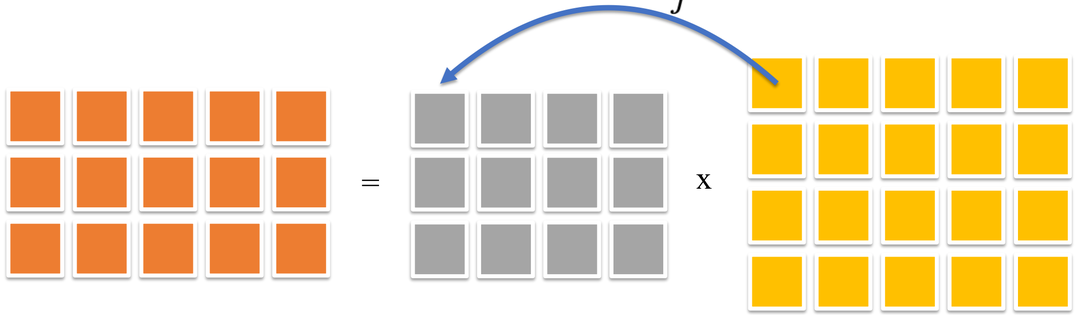
Axbxc matmul cxd -> Axbxd



矩阵

- **乘法（矩阵 矩阵）**

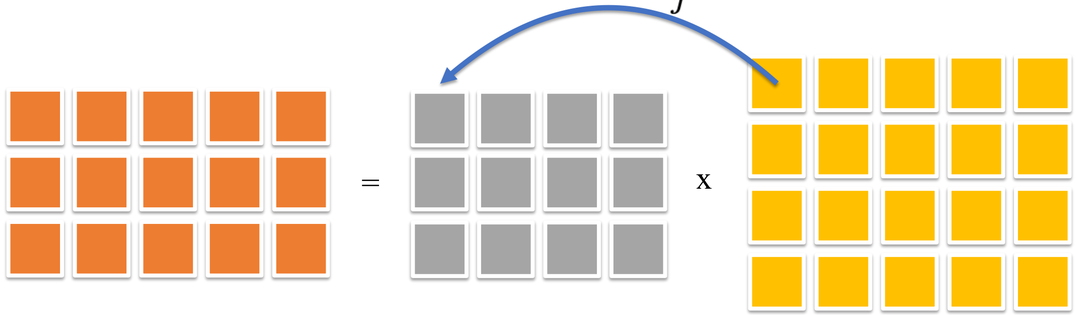
$$C = AB \text{ where } C_{ik} = \sum_j A_{ij} B_{jk}$$



Matrices

- **Multiplications (matrix matrix)**

$$C = AB \text{ where } C_{ik} = \sum_j A_{ij} B_{jk}$$



矩阵

- **长度（范数）**

$$||B||_{Frob} \leq \left[\sum_{ij} B_{ij}^2\right]^{\frac{1}{2}}$$

$$||B - L||_{Frob} \sum |B||_{Frob} - ||L||_{Frob}$$
$$||eB||_{Frob} \leq |e| A||B||_{Frob}$$

其他范数：p-范数、核范数、谱范数

.....https://en.wikipedia.org/wiki/Matrix_范数

Matrices

- **Length (Norm)**

$$||A||_{Frob} = \left[\sum_{ij} A_{ij}^2\right]^{\frac{1}{2}}$$

$$||A + B||_{Frob} \leq ||A||_{Frob} + ||B||_{Frob}$$
$$||aA||_{Frob} = |a| \cdot ||A||_{Frob}$$

Other norms: p-norm, nuclear norm, spectral norm…

https://en.wikipedia.org/wiki/Matrix_norm

图像分类

K 最近邻



Image Classification

K Nearest Neighbor



问题



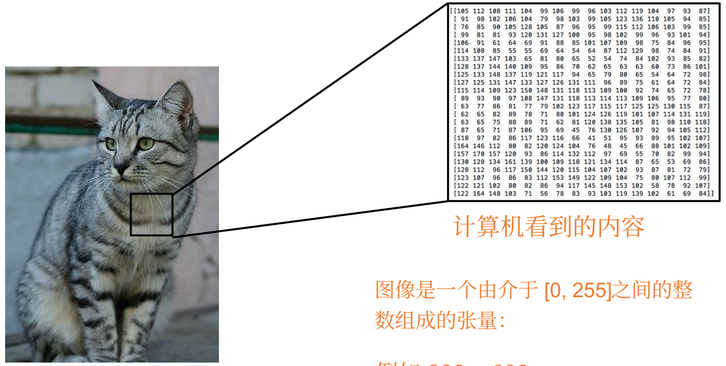
Cat

Problem



Cat

问题

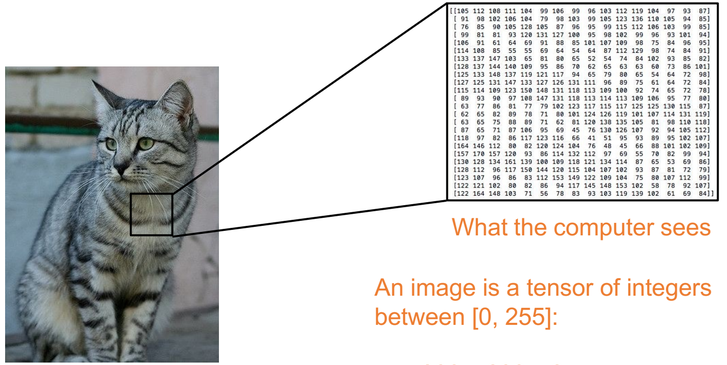


计算机看到的内容

图像是一个由介于 [0, 255]之间的整数组成的张量：

例如 800 x 600 x 3（3 通道 RGB）

Problem

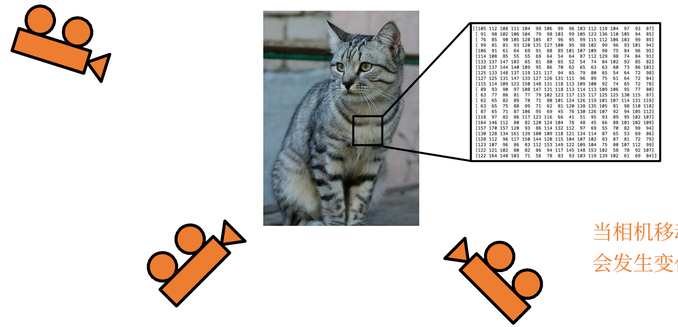


What the computer sees

An image is a tensor of integers between [0, 255]:

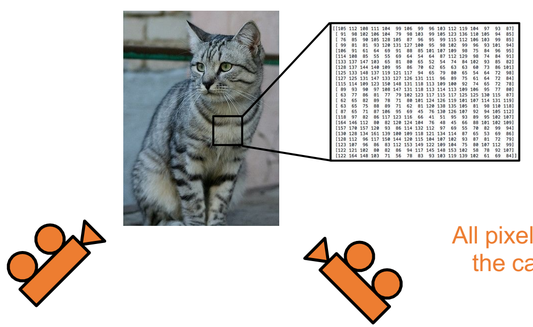
e.g. 800 x 600 x 3
(3 channels RGB)

挑战



当相机移动时，所有像素都会发生变化！

Challenges



All pixels change when the camera moves!

挑战：背景杂乱




Challenges: Background Clutter




挑战：光照



Challenges: illumination



挑战：遮挡



Challenges: Occlusion



挑战：变形



Challenges: Deformation



挑战：类内差异



Challenges: Intraclass variation



我们如何对图像进行分类?

```
def classify_image(image):
  # Some magic here?
  return class_label
```

不同于例如对数字列表进行排序,

没有明显的方法来硬编码用于识别猫或其他类别的算法。



How do we classify an image?

```
def classify_image(image):
  # Some magic here?
  return class_label
```

Unlike e.g. sorting a list of numbers,

no obvious way to hard-code the algorithm for recognizing a cat, or other classes.

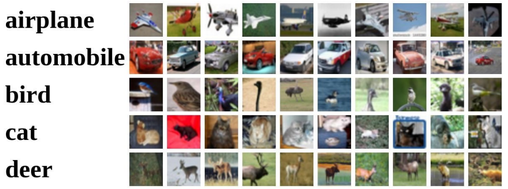


我们如何对图像进行分类?

1. 收集图像和标签的数据集 2. 提取图像的判别性特征 3. 使用机器学习算法训练分类器 4. 在新图像上评估分类器

```
def train(images, labels):
  # Machine learning!
  return model
```

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

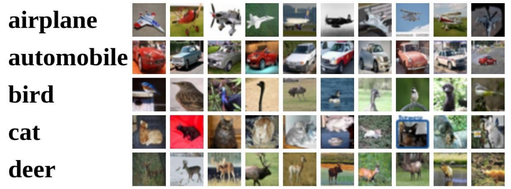


How do we classify an image?

1. Collect a dataset of images and labels
2. Extract discriminative features of the images
3. Use machine learning algorithms to train a classifier
4. Evaluate the classifier on new images

```
def train(images, labels):
  # Machine learning!
  return model
```

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



最近邻

```
def train(images, labels):
  # Machine learning!
  return model
```

记忆所有数据和标签

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

预测最相似训练图像的标签

Nearest neighbor

```
def train(images, labels):
  # Machine learning!
  return model
```

Memorize all data and labels

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

Predict the label of the most similar training image

最近邻



带标签的训练数据



查询图像

距离度量 $\left| \quad , \quad \right| \rightarrow R$

Nearest neighbor



Training data with labels



query image

Distance Metric $\left| \quad , \quad \right| \rightarrow R$

Distance Metric 用于比较图像

L1 距离： $d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$

test image					training image					pixel-wise absolute value differences				
56	32	10	18	-	10	20	24	17	=	46	12	14	1	add → 456
90	23	128	133		8	10	89	100		82	13	39	33	
24	26	178	200		12	16	178	170		12	10	0	30	
2	0	255	220		4	32	233	112		2	32	22	108	

Distance Metric to compare images

L1 distance: $d_1(I_1, I_2) = \sum_p |I_1^p - I_2^p|$

test image					training image					pixel-wise absolute value differences				
56	32	10	18	-	10	20	24	17	=	46	12	14	1	add → 456
90	23	128	133		8	10	89	100		82	13	39	33	
24	26	178	200		12	16	178	170		12	10	0	30	
2	0	255	220		4	32	233	112		2	32	22	108	

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

最近邻分类器


```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```


Nearest Neighbor classifier

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

最近邻分类器

记忆训练数据


```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```


Nearest Neighbor classifier

Memorize training data

```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```

最近邻分类器

对于每张测试图像：找到最接近的训练图像，预测最近图像的标签


```python
import numpy as np

class NearestNeighbor:
  def __init__(self):
    pass

  def train(self, X, y):
    """ X is N x D where each row is an example. Y is 1-dimension of size N """
    # the nearest neighbor classifier simply remembers all the training data
    self.Xtr = X
    self.ytr = y

  def predict(self, X):
    """ X is N x D where each row is an example we wish to predict label for """
    num_test = X.shape[0]
    # lets make sure that the output type matches the input type
    Ypred = np.zeros(num_test, dtype = self.ytr.dtype)

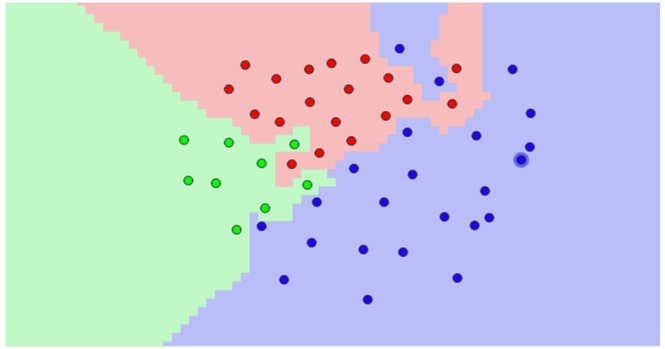
    # loop over all test rows
    for i in xrange(num_test):
      # find the nearest training image to the i'th test image
      # using the L1 distance (sum of absolute value differences)
      distances = np.sum(np.abs(self.Xtr - X[i,:]), axis = 1)
      min_index = np.argmin(distances) # get the index with smallest distance
      Ypred[i] = self.ytr[min_index] # predict the label of the nearest example

    return Ypred
```


Nearest Neighbor classifier

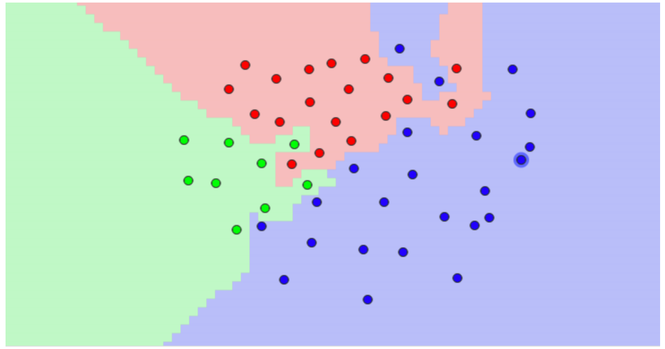
For each test image:
Find closest train image
Predict label of nearest image

这看起来像什么?



1-最近邻

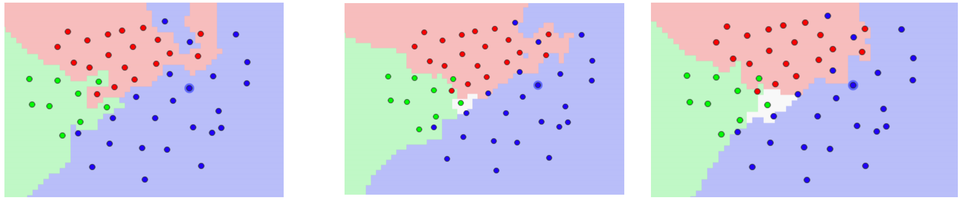
What does this looks like?



1-nearest neighbor

从最近邻到K近邻

不从最近邻复制标签，而是从 K 个最近点中取**多数投票**

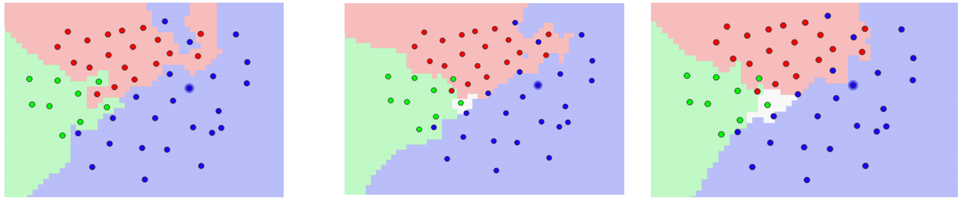


K r =　　K r]　　K r 5

一个交互式演示：https://lecture-demo.ira.uka.de/knn-demo/presetc=

From Nearest neighbor to K Nearest Neighbor

Instead of copying label from nearest neighbor, take **majority vote** from K closest points



K = 1　　K = 3　　K = 5

An interative demo: https://lecture-demo.ira.uka.de/knn-demo/preset=0

超参数很重要！

要使用的最佳 □值是多少？ 要使用的最佳 **距离** 是什么？

这些是**超参数**：关于算法本身的选择。

非常依赖于具体问题和数据集。
必须把它们全部尝试一遍，看看哪种效果最好。



Hyperparameters matters!

What is the best value of **k** to use? What is the best **distance** to use?

These are **hyperparameters**: choices about the algorithms themselves.

Very problem/dataset-dependent.
Must try them all out and see what works best.



设置超参数

想法 #1：选择在 **训练数据** 上效果最好的超参数

□□□：Ｋｒ＝在训练数据上总是表现完美



Setting hyperparameters

Idea #1: Choose hyperparameters that work best on the **training data**

BAD: K = 1 always works perfectly on training data



设置超参数

想法 #1：选择在**训练数据**上表现最佳的超参数

□□□：K r = 总是能在训练数据上完美运行



思路#2：选择在**测试**数据上表现最佳的超参数

□□□：不知道算法如何在新数据上的表现



Setting hyperparameters

Idea #1: Choose hyperparameters that work best on the **training data**

BAD: K = 1 always works perfectly on training data



Idea #2: choose hyperparameters that work best on **test** data

BAD: No idea how algorithm will perform on new data



设置超参数

想法 #1：选择在**训练数据**上表现最佳的超参数

糟糕：Ｋｒ＝在训练数据上总是表现完美



思路#2：选择在**测试**数据上表现最佳的超参数

糟糕：不知道算法如何将在新数据上表现



想法 #3：将数据划分为 **训练**集、**验证**集；在验证集上选择超参数，并在测试集上评估

更好！



Setting hyperparameters

Idea #1: Choose hyperparameters that work best on the **training data**

BAD: K = 1 always works perfectly on training data



Idea #2: choose hyperparameters that work best on **test** data

BAD: No idea how algorithm will perform on new data



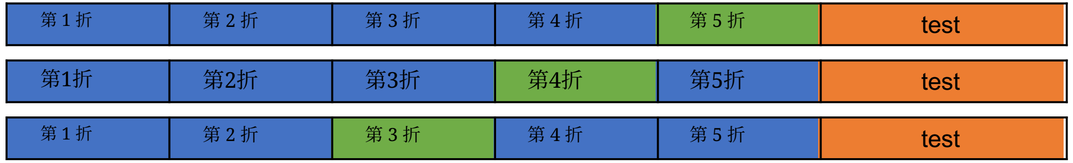
Idea #3: Split data into **train**, **val**; choose hyperparameters on val and evaluate on test

Better!



设置超参数

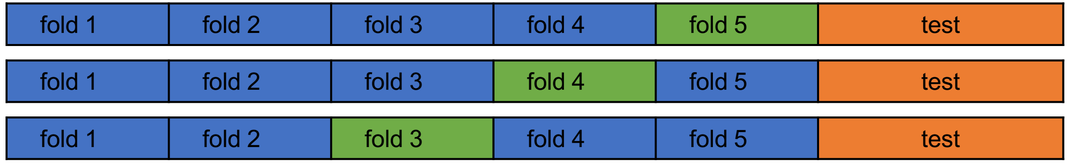
想法 #4：**交叉验证**：将数据分成多个 **折**，依次将每一折作为验证集，并对结果取平均



对小型数据集有用，但在深度学习中并不太常用

Setting hyperparameters

Idea #4: **Cross-Validation**: Split data into **folds**, try each fold as validation and average the results



Useful for small datasets, but not used too frequently in deep learning

KNN 在 **CIFAR10** 中的结果

DB 个类别□**BBBB** 张训练图像**DBBBB**张测试图像



Results of KNN in **CIFAR10**

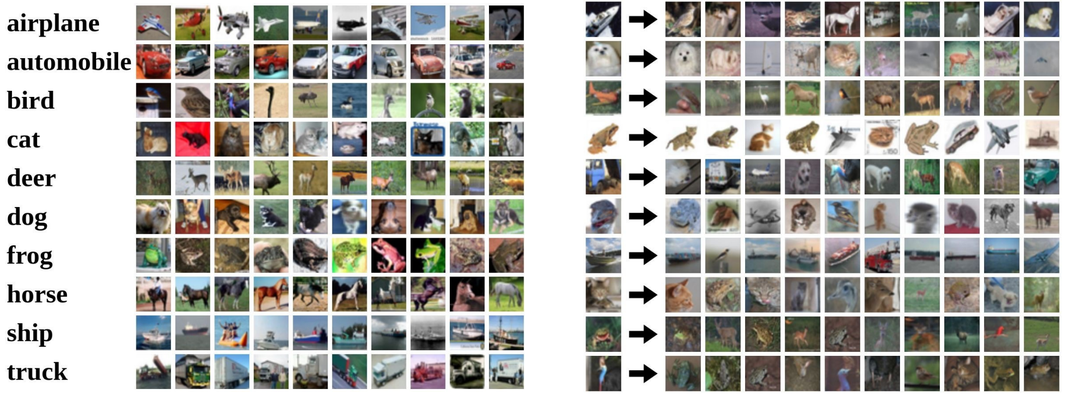
10 classes
50,000 training images
10,000 testing images



KNN 在 **CIFAR10** 中的结果

DB 个类别□**BBBB** 张训练图像**DBBBB**张测试图像

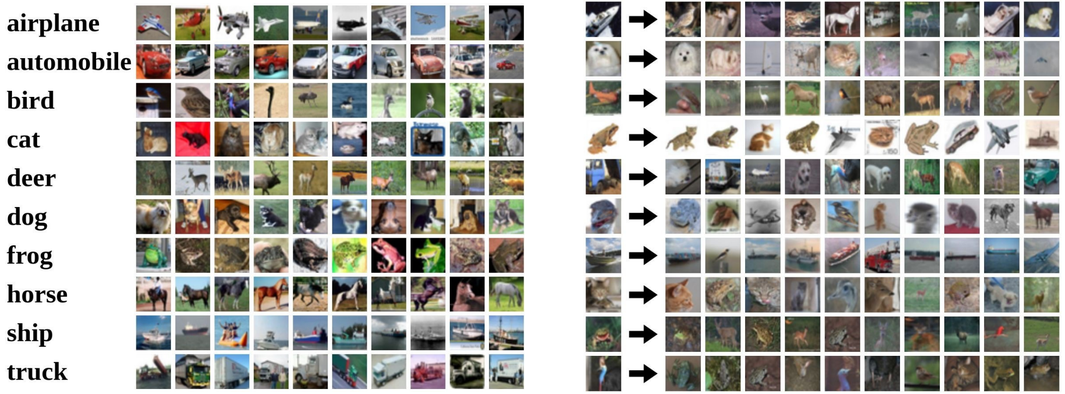
使用原始像素特征和最近邻的测试图像



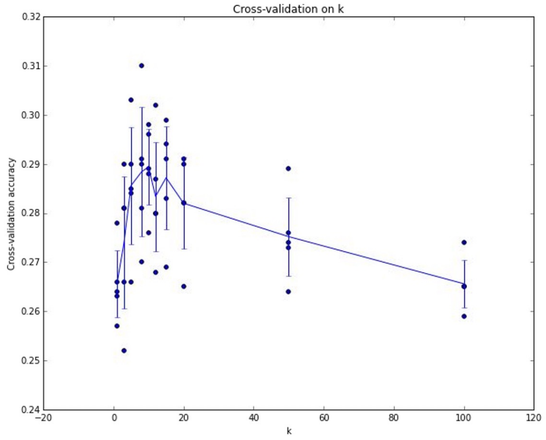
Results of KNN in **CIFAR10**

10 classes
50,000 training images
10,000 testing images

Test images with raw pixel feature and nearest neighbors



设置超参数



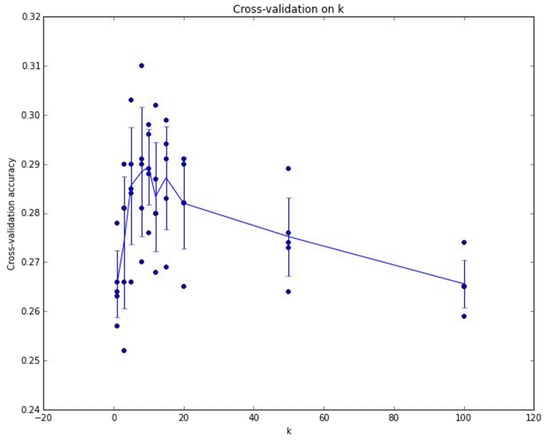
用于 **k.** 值的 5 折交叉验证示例

每个点：单个结果。

该线穿过均值，条形表示标准差。

（似乎 k □□ □ 对这些数据效果最好）

Setting hyperparameters



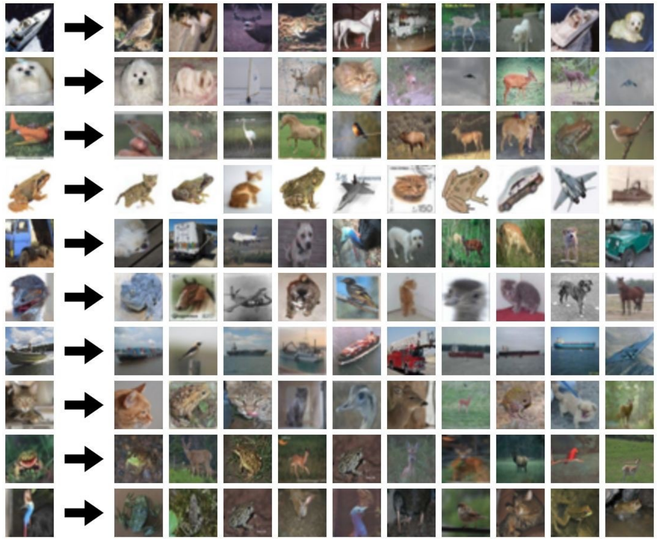
Example of
5-fold cross-validation for the value of **k.**

Each point: single outcome.

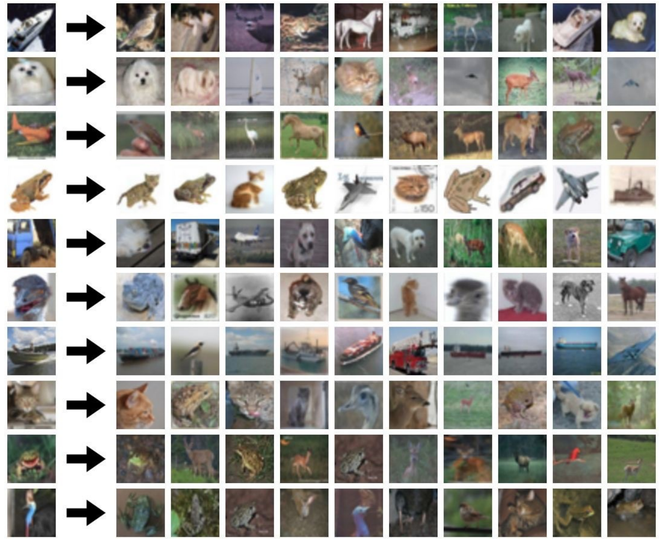
The line goes
through the mean, bars indicated standard deviation

(Seems that k ~= 7 works best for this data)

这看起来像什么?



What does this look like?



这看起来像什么?



What does this look like?



使用像素距离的 k 近邻**从未被使用**。

- 基于像素的距离度量信息量不足 - 测试时非常慢

需要进行特征提取！



原文　遮挡　打乱（1 像素）　重新着色

k-Nearest Neighbor with pixel distance **never used.**

- Distance metrics on pixels are not informative
- Very slow at test time

Feature extraction is needed!



Original　Occluded　Shuffled (1 pixel)　Recolored

我们如何对图像进行分类?



特征提取

机器学习

How do we classify an image?



Feature Extraction

Machine learning

我们如何对图像进行分类?



特征提取 → 机器学习

原始像素 方向梯度直方图（HoG） □1□局部二值模式（LBP） □2□尺度不变特征变换（SIFT） □3□

□

[1]“用于人体检测的方向梯度直方图。” CVPR 2015[□] “用于纹理分类的局部二值模式算子的完整建模”, TIP2010[□]“ ”SIFT：预测影响蛋白质功能的氨基酸变化。“ Nucleic acids research, 2013

How do we classify an image?



Feature Extraction → Machine learning

Raw pixels
Histogram of Oriented Gradient (HoG) [1]
Local Binary Pattern (LBP) [2]
Scale Invariant Feature Transform (SIFT) [3]
…

[1] “Histograms of oriented gradients for human detection.” CVPR 2015
[2] “A completed modeling of local binary pattern operator for texture classification”, TIP 2010
[3] “ ”SIFT: Predicting amino acid changes that affect protein function.“ Nucleic acids research, 2013

我们如何对图像进行分类?



特征提取

机器学习

最近邻、线性分类器、
支持向量机.....

How do we classify an image?



Feature Extraction

Machine learning

Nearest Neighbor
Linear Classifier
Support Vector Machine
…

简要总结

- 在**图像分类** 中，我们从一个由图像和标签组成的**训练集** 开始，并且必须在**测试集**上预测标签
- **K-最近邻** 分类器根据 K 个最近的训练样本预测标签

- 距离度量和 K 是**超参数**

使用**验证集**选择超参数；

只在最后阶段在测试集上运行一次！

A Brief Summary

- In **image classification** we start with a **training set** of images and labels, and must predict labels on the **test set**
- The **K-Nearest Neighbors** classifier predicts labels based on the K nearest training examples

- Distance metric and K are **hyperparameters**

Choose hyperparameters using the **validation set**;

Only run on the test set once at the very end!

线性分类器

Linear Classifier

一种更典型的框架

K 最近邻

```
def train(images, labels):
  # Machine learning!
  return model
```

记住所有数据和标签

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

预测最相似训练图像的标签

A more typical framework

K Nearest Neighbor

```
def train(images, labels):
  # Machine learning!
  return model
```

Memorize all data and labels

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

Predict the label of the most similar training image

更典型的框架

线性分类器

```
def train(images, labels):
  # Machine learning!
  return model
```

定义模型 $f_W(x)$ 定义损失函数 L $f_W(x), y$ 求解使 L 最小化的 W^*

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

根据 $f_{W^*}(x)$ 预测标签

A more typical framework

Linear classifier

```
def train(images, labels):
  # Machine learning!
  return model
```

Define a model $f_W(x)$
Define a loss function $L(f_W(x), y)$
Solve W^* that leads to minimal L

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

Predict the labels based on $f_{W^*}(x)$

一种更典型的框架

线性分类器

```
def train(images, labels):
  # Machine learning!
  return model
```

定义模型 $f_W(x)$定义损失函数
L $f_W(x)$, y求解 W^* 以最小化 L

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

基于 $f_{W^*}(x)$ 预测标签

A more typical framework

Linear classifier

```
def train(images, labels):
  # Machine learning!
  return model
```

Define a model $f_W(x)$
Define a loss function $L(f_W(x), y)$
Solve W^* that leads to minimal L

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```

Predict the labels based on $f_{W^*}(x)$

问题形式化



$I \in R^{32\times32\times3}$



$x \in R^{3072\times1}$

向量化

$W(□, b, x)$

W, b

参数

□□ 个数字给出类别得分

预测：最大得分

Problem formulation



$I \in R^{32\times32\times3}$



$x \in R^{3072\times1}$

vectorization

$f(x, W, b)$

W, b

parameters

10 numbers giving class scores

Prediction: largest score

问题形式化



我 $\in R^{32\times32\times3}$



$\boldsymbol{x} \in R^{3073\times1}$

向量化

$W(?, \boldsymbol{b})$

参数

□□ 个数字给出类别得分

预测：最大分数

通过增强该特征，我们可以消除偏差。

Problem formulation



I $\in R^{32\times32\times3}$



$\boldsymbol{x} \in R^{3073\times1}$

vectorization

$f(\boldsymbol{x}, \boldsymbol{W})$

parameters

10 numbers giving class scores

Prediction: largest score

By augmenting the feature, we can remove the bias.

一个更典型的框架

线性分类器

```
def train(images, labels):
  # Machine learning!
  return model
```



定义模型 $f_W(x)$定义损失函数 L $f_W(x)$, y求解 W^*，使 L最小

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



基于 $f_{W^*}(x)$ 预测标签

A more typical framework

Linear classifier

```
def train(images, labels):
  # Machine learning!
  return model
```



Define a model $f_W(x)$
Define a loss function $L(f_W(x), y)$
Solve W^* that leads to minimal L

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



Predict the labels based on $f_{W^*}(x)$

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b



猫 **3.2** 1.3 2.2汽车 5.1

4.92.5青蛙 -1.7 2.0**-3.1**

给定一个由样本组成的数据集

$\{x_i, y_i\}_{i=1}^{N}$

其中 x_i 是图像，y_i 是标签。

$$L = \frac{1}{N}\sum_i L_i(f(x_i, \boldsymbol{W}), y_i)$$

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & o\lambda rpysisp \end{cases}$$

其中 $\boldsymbol{s} = f\ x_i, \boldsymbol{W}\ \in R^3$ 是预测分数。

Solving $\boldsymbol{w}$ and b with mult-class SVM loss



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

Given a dataset of examples

$\{x_i, y_i\}_{i=1}^{N}$

Where x_i is the image and y_i is the label.

$$L = \frac{1}{N}\sum_i L_i(f(x_i, \boldsymbol{W}), y_i)$$

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

where $\boldsymbol{s} = f(x_i, \boldsymbol{W}) \in R^3$ is the prediction score.

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b



猫 **3.2** 1.3 2.2汽车 5.1

4.92.5青蛙 -1.7 2.0**-3.1**

这意味着什么?

多类 SVM 损失:

$$L_i = \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & o\lambda\ pysisp \end{cases}$$

其中 $\boldsymbol{s} = f\ x_i、\ \boldsymbol{W}\ \in R^3$ 是预测分数。

Solving $\boldsymbol{w}$ and b with mult-class SVM loss



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

What does it mean?

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

where $\boldsymbol{s} = f(x_i, \boldsymbol{W}) \in R^3$ is the prediction score.

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

多类 SVM 损失：

$$L_i = \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & o\lambda\ pysisp \end{cases}_{\text{h}}$$

对于除 targetone 之外的其他类别所对应的所有分数

Solving $\boldsymbol{w}$ and b with mult-class SVM loss



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

For all scores corresponding to classes other than target one

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

多类 SVM 损失：

$$L_i = \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & o\lambda\ pysisp \end{cases}$$ h

如果得分值小于目标类别的得分值加上一个间隔

3.7 n-1.a - 5

则损失为 0

Solving $\boldsymbol{w}$ and b with mult-class SVM loss



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

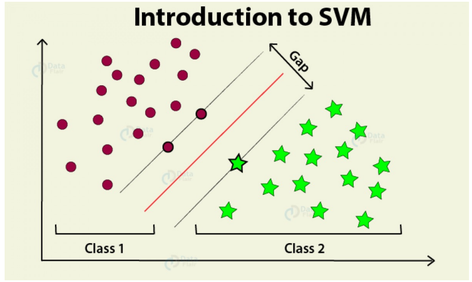
If the score value is smaller than that of target class plus a margin

3.2 > -1.7 + 1

Then the loss is 0

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b

间隔：我们希望促使每两个类别之间由一个间隙分隔。



来源：https://morioh.com/p/74f4a4b78e0b

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & o\lambda rpysisp \end{cases}$$

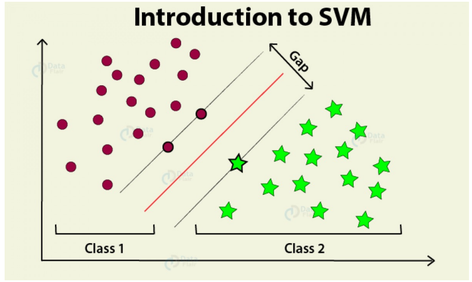
如果分数值小于目标类别的值加上一个 间隔

<.7 n 15.a - 5

那么损失为 0

Solving $\boldsymbol{w}$ and b with mult-class SVM loss

Margin: we want to encourage each two classes be separated by a gap.



source: https://morioh.com/p/74f4a4b78e0b

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

If the score value is smaller than that of target class plus a margin

3.2 > -1.7 + 1

Then the loss is 0

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & o\lambda\ pysisp \end{cases}$$

h

否则

3.7 1 >5 - 5

损失是当前得分与目标得分之差再加上一个间隔

Solving $\boldsymbol{w}$ and b with mult-class SVM loss



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

Otherwise

3.2 < 5.1 + 1

The loss is the difference between the current and target score plus a margin

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b



猫 **3.2** 1.3 2.2汽车 5.1

4.92.5青蛙 -1.7 2.0**-3.1**

多类 SVM 损失：

$$L_i = \begin{matrix} . \\ 0 \\ s_j - s_{y_i} + 1 \end{matrix} \begin{matrix} j \neq y_i / \\ if\ s_{y_i} > s_j + 1 \\ o\lambda\ pysisp \end{matrix}$$ 小时

重写

$$L_i = \cdot \max(0, s_j - s_{y_i} + 1)$$

$$j \neq y_i$$

Solving $\boldsymbol{w}$ and b with mult-class SVM loss



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \begin{cases} 0 & if\ s_{y_i} > s_j + 1 \\ s_j - s_{y_i} + 1 & othersise \end{cases}$$

Rewrite

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

利用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b

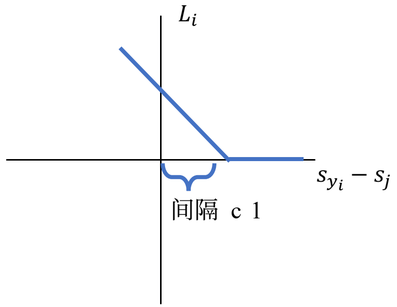


猫 **3.2** 1.3 2.2汽车 5.1

4.92.5青蛙 -1.7 2.0**-3.1**

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \text{o g, } \left(0, s_j - s_{y_i} + 1\right)$$



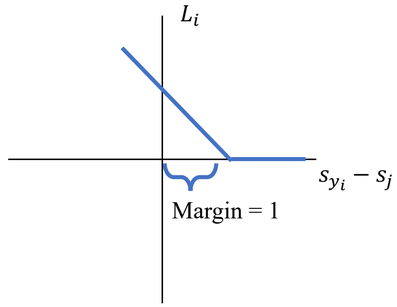
Solving $\boldsymbol{w}$ and b with mult-class SVM loss



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \max\left(0, s_j - s_{y_i} + 1\right)$$



使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
□oss	2.9		

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \text{o g, } \left(0, s_j - s_{y_i} + 1\right)$$

~ max(0, 5.1 - 3.2 3 1) 3
max(0, -1.7 - 3.2 3 1) ~
max(0, 2.9) 3 max(0, -
3.9) ~ 2.t 3 L~ 2.9

Solving $\boldsymbol{w}$ and b with mult-class SVM loss



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
Loss	2.9		

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \max\left(0, s_j - s_{y_i} + 1\right)$$

= max(0, 5.1 - 3.2 + 1)
+max(0, -1.7 - 3.2 + 1)
= max(0, 2.9) + max(0, -3.9)
= 2.9 + 0
= 2.9

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
□oss	2.9	0	

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \text{o g, } (0, s_j - s_{y_i} + 1)$$

~ max(0, 1.3 - 4.t 3 1) 3 max(0, 2.0 - 4.t 3 1) ~ max(0, -2.6) 3 max(0, -1.9) ~ L 3 L~ L

Solving $\boldsymbol{w}$ and b with mult-class SVM loss



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
Loss	2.9	0	

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

= max(0, 1.3 - 4.9 + 1)
+max(0, 2.0 - 4.9 + 1)
= max(0, -2.6) + max(0, -1.9)
= 0 + 0
= 0

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
□oss	2.9	0	12.9

多类SVM损失：

$$L_i = \sum_{j \neq y_i} \text{o g, } \left(0, s_j - s_{y_i} + 1\right)$$

~ max(0, 2.2 - (-3.1) 3 1)3
max(0, 2.5 - (-3.1) 3 1)~
max(0, 6.3) 3 max(0,
6.6)~ r .f 3 r .6~ 12.9

Solving $\boldsymbol{w}$ and b with mult-class SVM loss



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
Loss	2.9	0	12.9

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \max\left(0, s_j - s_{y_i} + 1\right)$$

= max(0, 2.2 - (-3.1) + 1)
+max(0, 2.5 - (-3.1) + 1)
= max(0, 6.3) + max(0, 6.6)
= 6.3 + 6.6
= 12.9

使用多类 SVM 损失求解 $\boldsymbol{w}$ 和 b



猫 **3.2** 1.3 2.2汽车 5.1

4.92.5青蛙 -1.7 2.0**-3.1**损失

2.9 0 12.9

多类 SVM 损失：

$$L_i = \sum_{j \neq y_i} \text{o g, } (0, s_j - s_{y_i} + 1)$$

$$L = \frac{1}{e} \sum_{i=1}^{N} L_i$$

$L = (2.9 + \infty + -2.$

$9 \quad C)/.$

$= \leq \approx \sum$

Solving $\boldsymbol{w}$ and b with mult-class SVM loss



cat	**3.2**	1.3	2.2
car	5.1	**4.9**	2.5
frog	-1.7	2.0	**-3.1**
Loss	2.9	0	12.9

Multiclass SVM loss:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1)$$

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i$$

$L = (2.9 + 0 + 12.9)/3$

$= \mathbf{5.27}$

示例代码

$$L_i = \sum_{j \neq y_i} \text{o g, } \left(0, s_j - s_{y_i} + 1\right)$$

```
def L_i_vectorized(x, y, W):
  scores = W.dot(x)
  margins = np.maximum(0, scores - scores[y] + 1)
  margins[y] = 0
  loss_i = np.sum(margins)
  return loss_i
```



Example code

$$L_i = \sum_{j \neq y_i} \max\left(0, s_j - s_{y_i} + 1\right)$$

```
def L_i_vectorized(x, y, W):
  scores = W.dot(x)
  margins = np.maximum(0, scores - scores[y] + 1)
  margins[y] = 0
  loss_i = np.sum(margins)
  return loss_i
```



最优的W唯一吗?

$$f(x; W) = Wx$$

$$L = \frac{\rightarrow}{f}\sum_{i=1}^{N}\sum_{j\neq y_j} \text{o a0}\left(\propto 1 y(x_i 25\ )_j \text{ } y(x_i 25\ )_{y_i} + \rightarrow\right)$$

假设我们找到了一个 **W**， 使得 L□⊨。这个 **W** 唯一吗?

不！ nW 也会导致 L□⊨。

我们如何在 W 和 nW 之间进行选择?

Is the optimal W unique?

$$f(x; W) = Wx$$

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_j} \max\left(0, f(x_i; W)_j - f(x_i; W)_{y_i} + 1\right)$$

Suppose we found a **W** such that L=0.
Is this **W** unique?

No! nW also lead to L=0.

How do we choose between W and nW?

最优的W是否唯一?

$$f(x; \boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$

$$L = \frac{\rightarrow}{f}\sum_{i=1}^{N}\sum_{j\neq y_j} \text{o a0}\left(\infty 1[\mathbf{5}\ \boldsymbol{x}]_j \text{ } [\mathbf{5}\ \boldsymbol{x}]_{y_i} + \rightarrow\right)$$

Is the optimal W unique?

$$f(x; \boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_j} \max\left(0, [\boldsymbol{W}\boldsymbol{x}]_j - [\boldsymbol{W}\boldsymbol{x}]_{y_i} + 1\right)$$

最优的W是否唯一?

$$f(\boldsymbol{x}; \boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$

$$L = \frac{\rightarrow}{f}\sum_{i=1}^{N}\sum_{j\neq y_j} \text{o a0}\left(\propto 1[\boldsymbol{W}\boldsymbol{5}\ \boldsymbol{x}]_j \text{ } ? \text{ } [\boldsymbol{W}\boldsymbol{5}\ \boldsymbol{x}]_{y_i} + \rightarrow\right)$$

Is the optimal W unique?

$$f(\boldsymbol{x}; \boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$

$$L = \frac{1}{N}\sum_{i=1}^{N}\sum_{j\neq y_j} \max\left(0, [n\boldsymbol{W}\boldsymbol{x}]_j - [n\boldsymbol{W}\boldsymbol{x}]_{y_i} + 1\right)$$

最优的$\boldsymbol{W}$是否唯一?

$$f(\boldsymbol{x};\boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$

$$L = \frac{\boldsymbol{W}}{f}\sum_{i=1}^{N}\sum_{j\neq y_j} \text{o a0}\left(\infty 1[\mathbf{5}\ \boldsymbol{x}]_j \text{ } [\mathbf{5}\ \boldsymbol{x}]_{y_i} + \frac{\rightarrow}{\boldsymbol{W}}\right)$$

随着 n 增加，间隔会变小，这可能导致过拟合。

$$f(x;W) = Wx$$

Is the optimal $\boldsymbol{W}$ unique?

$$f(\boldsymbol{x};\boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$

$$L = \frac{n}{N}\sum_{i=1}^{N}\sum_{j\neq y_j} \max\left(0, [\boldsymbol{W}\boldsymbol{x}]_j - [\boldsymbol{W}\boldsymbol{x}]_{y_i} + \frac{1}{n}\right)$$

As n increases, the margin gets small, which could lead to overfitting.

$$f(x;W) = Wx$$

正则化

$$N \leq \frac{\sum}{R} \square_{i=1}^{N} N_i(+_i;\ \boldsymbol{x}), \lambda_i)$$

数据损失：模型预测应与训练数据匹配

Regularization

$$L = \frac{1}{N}\sum_{i=1}^{N} L_i(\boldsymbol{x_i}; \boldsymbol{W}), y_i)$$

Data loss: Model predictions should match training data

正则化

$$N \leq \frac{\sum}{R} \square^{N} \quad N_i \;\; y_i; \;\; \boldsymbol{x} \;\; , \lambda_i) \quad -\partial a(\boldsymbol{x} \;)$$

为什么这个术语有效?

数据损失：模型预测应与训练数据匹配

正则化：对模型的偏好。

示例：a2 正则化：$R \;\; W \;\; = \;\; W \; \& \;\; = \leftarrow_k W_k^2$

L_1 正则化：$R \;\; W \;\; = \;\; W \; (\;\; = \leftarrow_k W_k$

| |

Regularization

$$L = \frac{1}{N} \sum_{i=1}^{N} L_i(x_i; \boldsymbol{W}), y_i) \quad + \lambda R(\boldsymbol{W})$$

Why does this term work?

Data loss: Model predictions should match training data

Regularization: preference over the model.

Example:

L$_2$ regularization: $R(W) = \left|\left|W\right|\right|_2 = \sum_k W_k^2$

L_1 regularization: $R(W) = \left|\left|W\right|\right|_1 = \sum_k |W_k|$

正则化

a2 正则化： $R\ W\ =\ W\ \&\ =\leftarrow_k W_k^2 L_1$

正则化： $R\ W\ =\ W\ (\ =\leftarrow_k W_k\ \ \ |\ \ |$

1. “最大间隔”原理
在 SV 中 M，目标是找到一个能够分隔各个类别的决策边界 .
无正则化：模型可能会过度努力地正确分类训练数据中的每一个噪声点，从而导致非常复杂的边界。

使用正则化：最小化 $W\ _2$ 在数学上等价于 **最大化间隔**。更大的间隔通常会在新的、未见过的数据上带来更好的性能，因为它更稳健。

2. 控制“权重”
为什么权重要小？如果权重 W_i 非常大，那么特定输入特征 x_i 的微小变化也会导致巨大波动 g
在输出中。通过迫使权重变小，正则化确保模型不会过度依赖任何单一
特征 o 训练集中的 r 特定噪声。这会带来一个 **更平滑** 的模型。

Regularization

L_2 regularization: $R(W) = \left||W|\right|_2 = \sum_k W_k^2$

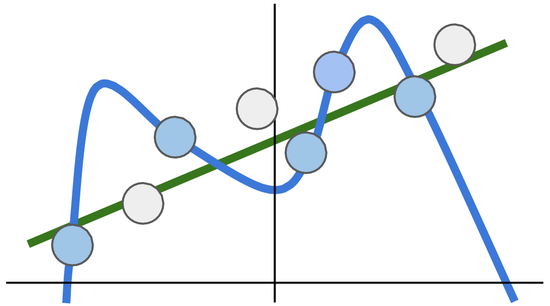
L_1 regularization: $R(W) = \left||W|\right|_1 = \sum_k |W_k|$

1. The "Maximum Margin" Principle
In an SVM, the goal is to find a decision boundary that separates classes.
Without Regularization: The model might try too hard to correctly classify every single noise point in the training data, leading to a very complex boundary.
With Regularization: Minimizing $\left||W|\right|_2$ is mathematically equivalent to **maximizing the margin.** A larger margin usually leads to better performance on new, unseen data because it’s more robust.

2. Controlling the "Weights"
Why small weights? If a weight W_i is very large, even a tiny change in a specific input feature x_i will cause a huge swing in the output. By forcing the weights to be small, regularization ensures that the model doesn't over-rely on any single feature or specific noise in the training set. This leads to a **smoother** model.

正则化

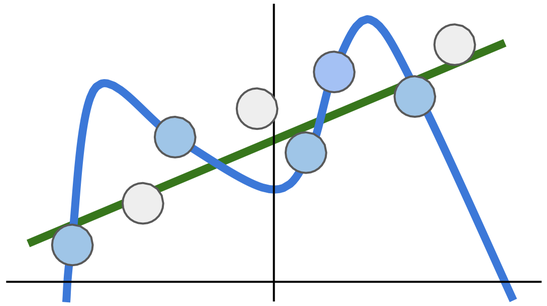
如果两个模型对数据的解释效果相同，那么通常更简单的那一个才是正确的!



Occam's Razar：在多个相互竞争的假设中，最简单的就是最好的，William of Ockham 1285-1347

Regularization

If two models explain the data equally well, the simpler one is usually the correct one!



Occam's Razar: Among multiple competing hypotheses, the simplest is the best，William of Ockham 1285-1347

其他更复杂的正则化方法

Dropout
- Srivastava, Nitish, 等。“Dropout：一种防止神经网络过拟合的简单方法。” JMLR, 2014
- 批归一化
  - Ioffe, Sergey, 和 Christian Szegedy。“批量归一化：通过减少内部协变量偏移加速深度网络训练。” ICML, 2015.

Mixup
- Zhang, Hongyi, 等. “mixup：超越经验风险最小化。”□*R*□□2018.

□

Other more complicated regularizations

- Dropout
  - Srivastava, Nitish, et al. “Dropout: a simple way to prevent neural networks from overfitting.” JMLR, 2014
- Batch normalization
  - Ioffe, Sergey, and Christian Szegedy. “Batch normalization: Accelerating deep network training by reducing internal covariate shift.” ICML, 2015.
- Mixup
  - Zhang, Hongyi, et al. “mixup: Beyond Empirical Risk Minimization.” *ICLR* 2018.

…

SVM 分类器的分数不具备概率可解释性

解决方案：Softmax 分类器！



0 The scores of the SVM classifier is not probabilistically interpretable.

Solution: Softmax classifier!



将预测得分转换为概率



概率必须为 □□ □

概率之和必须为 1

cat	**3.2**	ex□	**24.5**	归一化	**0.13**
car	5.1		164.0		0.87
frog	-1.7		0.18		0.00

$\boldsymbol{a} = y(x_i, m)$ $\quad x^s$ $\quad \frac{x^{s_k}}{\leftarrow_j x_{,j}}$

Softmax 函数

Converting prediction score to probability



Probabilities must be >= 0

Probabilities must sum to 1

cat	**3.2**	exp	**24.5**	normalize	**0.13**
car	5.1		164.0		0.87
frog	-1.7		0.18		0.00

$s = f(x_i, W)$ $\quad e^s$ $\quad \frac{e^{s_k}}{\sum_j e^{s_j}}$

Softmax function

损失函数

比较两个概率分布：交叉熵

$$L = -\frac{1}{e}\sum_i u_i \text{ml}\,(x_i) = -\frac{1}{e}\sum_i \text{ml}\left(\frac{p^{s_{y_i}}}{\leftarrow_j p^{s_j}}\right)$$



概率必须为 □□□

概率之和必须为1

cat	**3.2**	ex□	**24.5**	归一化	**0.13**	**1**
car	5.1		164.0		0.87	**0**
frog	-1.7		0.18		0.00	0

比较

$\boldsymbol{a} = y(x_i, m)$ $\quad x^s$ $\quad \frac{x^{s_k}}{\leftarrow_j x_{,j}}$

Loss function

Comparing two probability distributions:
Cross entropy

$$L = -\frac{1}{N}\sum_i p_i \log(q_i) = -\frac{1}{N}\sum_i \log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$



Probabilities must be >= 0

Probabilities must sum to 1

cat	**3.2**	exp	**24.5**	normalize	**0.13**	**1**
car	5.1		164.0		0.87	**0**
frog	-1.7		0.18		0.00	0

compare

$s = f(x_i, W)$ $\quad e^s$ $\quad \frac{e^{s_k}}{\sum_j e^{s_j}}$

损失函数

比较两个概率分布：
交叉熵

$$L = -\frac{1}{e}\sum_i u_i \ln(x_i) = -\frac{1}{e}\sum_i \ln\left(\frac{p^{s_{y_i}}}{\leftarrow_j p^{s_j}}\right)$$

问题1：交叉熵损失可能的最小/最大值是多少？
答：当 a_{y_i} W N 时，即所有预测都正确，那么

$$L \; ? \; ? \; \overrightarrow{f}\sum_i \text{mg}(\rightarrow) = \infty$$

当 a_{y_i} W ∞时，即所有预测都错误，那么

$$L \; ? \; ? \; \overrightarrow{f}\sum_i \log\infty \;\; W\,N$$

Loss function

Comparing two probability distributions:
Cross entropy

$$L = -\frac{1}{N}\sum_i p_i \log(q_i) = -\frac{1}{N}\sum_i \log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

Q1: What is the min/max possible cross entropy loss value?
A: When $s_{y_i} \to \infty$, i.e., all predictions are correct, then

$$L \approx -\frac{1}{N}\sum_i \log(1) = 0$$

When $s_{y_i} \to 0$, i.e., all predictions are wrong, then

$$L \approx -\frac{1}{N}\sum_i \log(0) \to \infty$$

损失函数

比较两个概率分布：
交叉熵

$$L = -\frac{1}{e}\sum_i u_i \log(x_i) = -\frac{1}{e}\sum_i \log\left(\frac{p^{s_{y_i}}}{\sum_j p^{s_j}}\right)$$

问题2：在初始化时，所有 a 都将近似相等；损失是多少？
答：假设有 C 个类别，

$$L = \frac{1}{f}\sum_i \log\left(\frac{1}{b}\right)$$

如果 Cg_M 那么 $L = \log(10) \approx 2.3$.

在调试模型时，请牢记这些值。

Loss function

Comparing two probability distributions:
Cross entropy

$$L = -\frac{1}{N}\sum_i p_i \log(q_i) = -\frac{1}{N}\sum_i \log\left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}}\right)$$

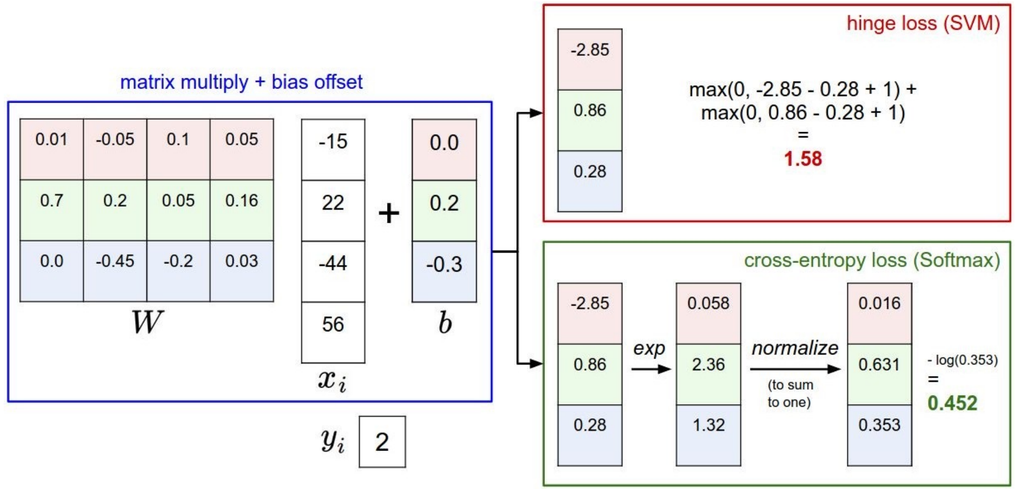
Q2: At initialization all s will be approximately equal; what is the loss?
A: Assume there is C classes,

$$L = -\frac{1}{N}\sum_i \log\left(\frac{1}{C}\right)$$

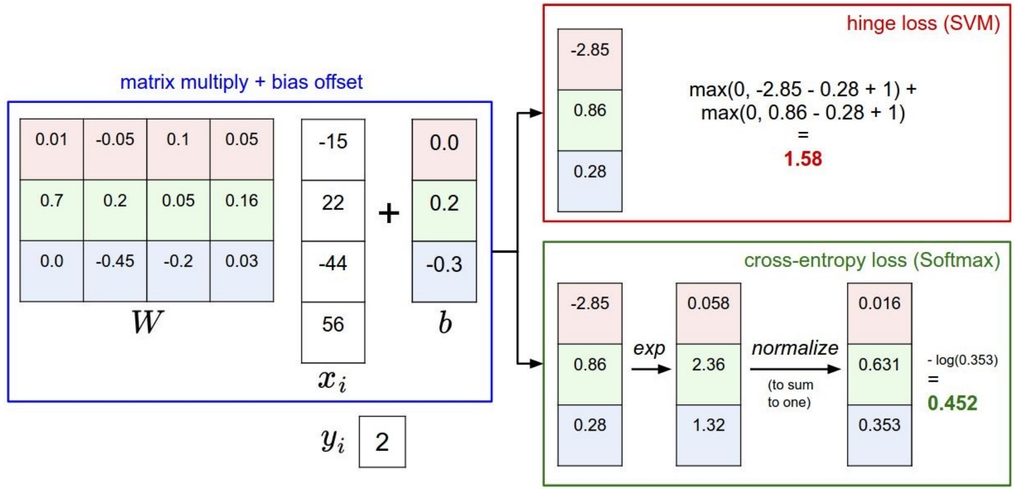
If C=10, then $L = \log(10) \approx 2.3$.

Keep these values in mind when you are debugging your model.

比较 Softmax 分类器与 SVM



Comparing Softmax classifier vs SVM



更典型的框架

线性分类器

```
def train(images, labels):
  # Machine learning!
  return model
```



定义一个模型 $f_W(x)$定义损失函数 L $f_W(x)$, y求解 W^* 从而最小化 L

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



基于 $f_{W^*}(x)$ 预测标签

A more typical framework

Linear classifier

```
def train(images, labels):
  # Machine learning!
  return model
```



Define a model $f_W(x)$
Define a loss function $L(f_W(x), y)$
Solve W^* that leads to minimal L

```
def predict(model, test_images):
  # Use model to predict labels
  return test_labels
```



Predict the labels based on $f_{W^*}(x)$

随机搜索?

```
# assume X_train is the data where each column is an example (e.g. 3073 x 50,000)
# assume Y_train are the labels (e.g. 1D array of 50,000)
# assume the function L evaluates the loss function

bestloss = float("inf") # Python assigns the highest possible float value
for num in xrange(1000):
  W = np.random.randn(10, 3073) * 0.0001 # generate random parameters
  loss = L(X_train, Y_train, W) # get the loss over the entire training set
  if loss < bestloss: # keep track of the best solution
    bestloss = loss
    bestW = W
  print 'in attempt %d the loss was %f, best %f' % (num, loss, bestloss)

# prints:
# in attempt 0 the loss was 9.401632, best 9.401632
# in attempt 1 the loss was 8.959668, best 8.959668
# in attempt 2 the loss was 9.044034, best 8.959668
# in attempt 3 the loss was 9.278948, best 8.959668
# in attempt 4 the loss was 8.857370, best 8.857370
# in attempt 5 the loss was 8.943151, best 8.857370
# in attempt 6 the loss was 8.605604, best 8.605604
# ... (truncated: continues for 1000 lines)
```



Random search?


```
# assume X_train is the data where each column is an example (e.g. 3073 x 50,000)
# assume Y_train are the labels (e.g. 1D array of 50,000)
# assume the function L evaluates the loss function

bestloss = float("inf") # Python assigns the highest possible float value
for num in xrange(1000):
  W = np.random.randn(10, 3073) * 0.0001 # generate random parameters
  loss = L(X_train, Y_train, W) # get the loss over the entire training set
  if loss < bestloss: # keep track of the best solution
    bestloss = loss
    bestW = W
  print 'in attempt %d the loss was %f, best %f' % (num, loss, bestloss)

# prints:
# in attempt 0 the loss was 9.401632, best 9.401632
# in attempt 1 the loss was 8.959668, best 8.959668
# in attempt 2 the loss was 9.044034, best 8.959668
# in attempt 3 the loss was 9.278948, best 8.959668
# in attempt 4 the loss was 8.857370, best 8.857370
# in attempt 5 the loss was 8.943151, best 8.857370
# in attempt 6 the loss was 8.605604, best 8.605604
# ... (truncated: continues for 1000 lines)
```




随机搜索?

```
# Assume X_test is [3073 x 10000], Y_test [10000 x 1]
scores = Wbest.dot(Xte_cols) # 10 x 10000, the class scores for all test examples
# find the index with max score in each column (the predicted class)
Yte_predict = np.argmax(scores, axis = 0)
# and calculate accuracy (fraction of predictions that are correct)
np.mean(Yte_predict == Yte)
# returns 0.1555
```

15.5% 准确率（SOTA 为 □t t .7%）

随机搜索非常慢

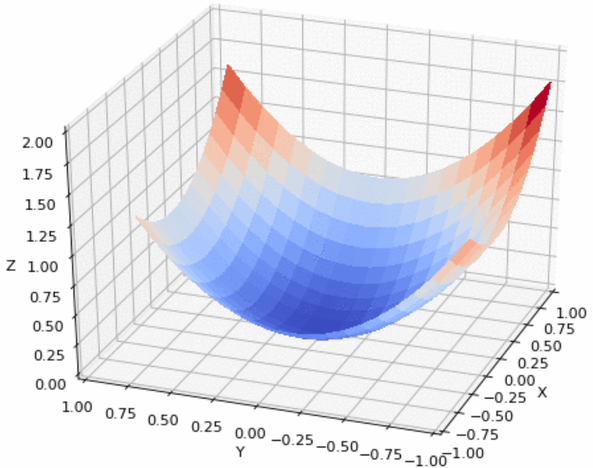
Random search?

```
# Assume X_test is [3073 x 10000], Y_test [10000 x 1]
scores = Wbest.dot(Xte_cols) # 10 x 10000, the class scores for all test examples
# find the index with max score in each column (the predicted class)
Yte_predict = np.argmax(scores, axis = 0)
# and calculate accuracy (fraction of predictions that are correct)
np.mean(Yte_predict == Yte)
# returns 0.1555
```

15.5% accuracy (SOTA is ~99.7%)

Random search is really slow

梯度下降

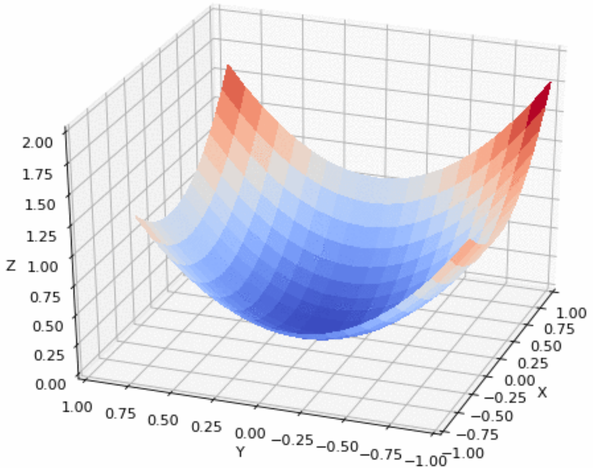


思路：沿着使损失下降最陡的方向前进。

$$W \in W - \frac{\partial L}{\partial W}$$

步长　　梯度

Gradient descent



Idea: follow the direction that leads to steepest loss reduction.

$$W \leftarrow W - \lambda \frac{\partial L}{\partial W}$$

Step size　　Gradient

梯度下降

$$\boldsymbol{x} \cdot \boldsymbol{x} = \partial \frac{\boldsymbol{W}N}{\boldsymbol{W}\boldsymbol{x}} \leq \boldsymbol{x} = \partial \frac{\boldsymbol{W}\frac{\sum}{R} \leftarrow_{i=1}^{N} N_i}{\boldsymbol{W}\boldsymbol{x}} \leq \boldsymbol{x} = \partial \frac{\sum}{R} \square_{i=1}^{N} \frac{\boldsymbol{W}N_i}{\boldsymbol{W}\boldsymbol{x}}$$

$$N_i \leq \square_{j \neq y_i} \max\left(1, \boldsymbol{x}_j^T y_i = \boldsymbol{x}_{y_i}^T y_i - \sum\right)$$

Gradient descent

$$\boldsymbol{W} \leftarrow \boldsymbol{W} - \lambda \frac{\partial L}{\partial \boldsymbol{W}} = \boldsymbol{W} - \lambda \frac{\partial \frac{1}{N} \sum_{i=1}^{N} L_i}{\partial \boldsymbol{W}} = \boldsymbol{W} - \lambda \frac{1}{N} \sum_{i=1}^{N} \frac{\partial L_i}{\partial \boldsymbol{W}}$$

$$L_i = \sum_{j \neq y_i} \max\left(0, \boldsymbol{W}_j^T x_i - \boldsymbol{W}_{y_i}^T x_i + 1\right)$$

梯度下降

$$N_i \leq \sum_{j \neq y_i} \max\left(1, \boldsymbol{x}_j^T y_i = \boldsymbol{x}_{y_i}^T y_i - \sum\right)$$

当 $>_j^T x_i - >_{y_i T} x_i + 1 \leq 0$ 时，则 $L_i = 0$，且梯度为 0

当 $>_j^T x_i - >_{y_i T} x_i + 1 > 0$ 时，则

$$\frac{\boldsymbol{x} L_i}{\boldsymbol{x5}_{y_i}} = -x_i \qquad \frac{\boldsymbol{x} L_i}{\boldsymbol{x5}_j} = x_i$$

Gradient descent

$$L_i = \sum_{j \neq y_i} \max\left(0, \boldsymbol{W}_j^T x_i - \boldsymbol{W}_{y_i}^T x_i + 1\right)$$

When $\boldsymbol{W}_j^T x_i - \boldsymbol{W}_{y_i}^T x_i + 1 \leq 0$, then $L_i = 0$, and the gradient is 0

When $\boldsymbol{W}_j^T x_i - \boldsymbol{W}_{y_i}^T x_i + 1 > 0$, then

$$\frac{\partial L_i}{\partial \boldsymbol{W}_{y_i}} = -x_i \qquad \frac{\partial L_i}{\partial \boldsymbol{W}_j} = x_i$$

损失函数

- 我们必须定义一种误差度量。
- 我们的模型有多不准?

- 均方误差
  - 对于给定的 x，求实际 y 值与预测 y 值（$y = mx + c$）之差。
  - 对该差值求平方。
  - 求 X 中每个值的平方的均值。

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - (mx_i + c))^2$$

Loss Function

- We must define a measure of error.
- How wrong is our model?

- Mean square error
  - Find the difference between the actual y and predicted y value ($y = mx + c$), for a given x.
  - Square this difference.
  - Find the mean of the squares for every value in X.

$$E = \frac{1}{n}\sum_{i=0}^{n}(y_i - (mx_i + c))^2$$

线性回归

- 监督学习的一项核心原则

寻找能够在所有可能数据上产生最小误差的模型参数。

- 我们只能观测到有限数量的数据，因此通常将其视为最小化训练数据上的误差（经验误差）。

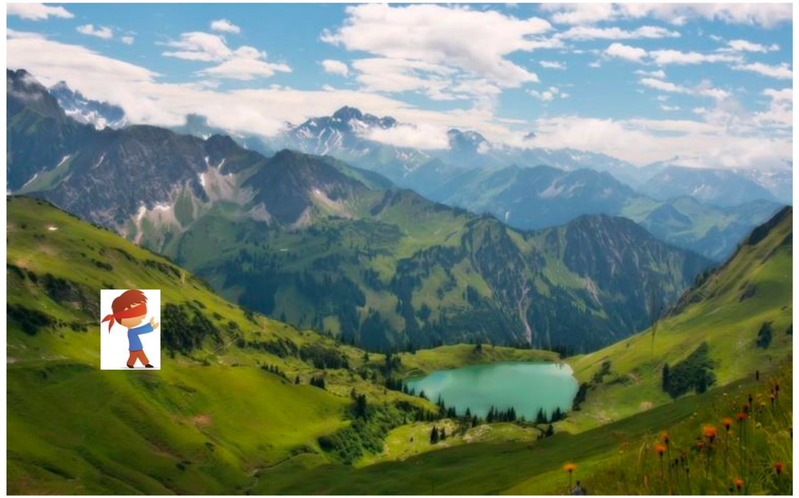
Linear Regression

- One central tenet of supervised learning

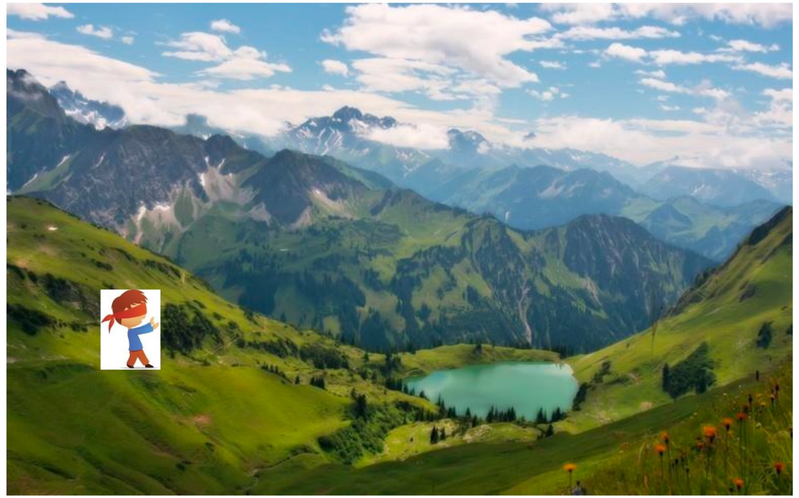
Find the model parameters that lead to smallest error on all possible data.

- We only observe limited amount of data, so it is usually taken as minimizing error on training data (empirical error).

优化



Optimization

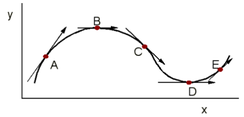


什么是梯度?

- 从几何上看
  - 指向函数增长最快的方向，其大小是该方向上图像的斜率

- 更形式化地说，在一维情况下

$$\frac{\partial f(x)}{\partial x} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$



- 在高维情况下

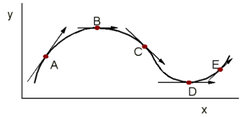
$$\frac{\partial f}{\partial x_i}(a_1, \dots, a_n) = \lim_{h \to 0} \frac{f(a_1, \dots, a_i + h, \dots, a_n) - f(a_1, \dots, a_i, \dots, a_n)}{h}$$

What is a gradient?

- Geometrically
  - Points in the direction of the greatest rate of increase of the function and its magnitude is the slope of the graph in that direction

- More formally in 1D

$$\frac{\partial f(x)}{\partial x} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h}$$



- In higher dimensions

$$\frac{\partial f}{\partial x_i}(a_1, \dots, a_n) = \lim_{h \to 0} \frac{f(a_1, \dots, a_i + h, \dots, a_n) - f(a_1, \dots, a_i, \dots, a_n)}{h}$$

数值梯度

- 可以将 h 设为一个非常小的数值并计算：

$$\frac{\partial f(x)}{\partial x} = \frac{f(x+h)-f(x)}{h}$$

- 速度慢，且仅是一种近似
  - 需要对每个参数计算一次得分
  - 对 h 的选择敏感

- h 也需要选择——超参数



Numeric gradient

- Can set h to a very low number and compute:

$$\frac{\partial f(x)}{\partial x} = \frac{f(x+h)-f(x)}{h}$$

- Slow and just an approximation
  - Need to compute score once for each parameter
  - Sensitive to choice of h

- h needs to be chosen as well - hyperparameter



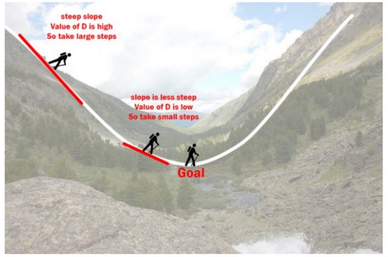
梯度下降

1. 初始时令 $\boldsymbol{D} = \infty$ 和 $c = \infty$ 令 L 为我们的学习率。它控制 $\boldsymbol{D}$ 的值在每一步变化多少。L 可以是一个较小的值，如 0.001，以获得良好的准确率。

2. 计算损失函数关于 $\boldsymbol{D}$ 的偏导数，并将 x、$\boldsymbol{c}$、$\boldsymbol{D}$ 和 c 的当前值代入其中，以获得导数值 $\boldsymbol{2}$。

$$D_m = \frac{1}{n}\sum_{i=0}^{n} 2(y_i - (mx_i + c))(-x_i)$$

$$D_m = \frac{-2}{n}\sum_{i=0}^{n} x_i(y_i - \bar{y}_i)$$

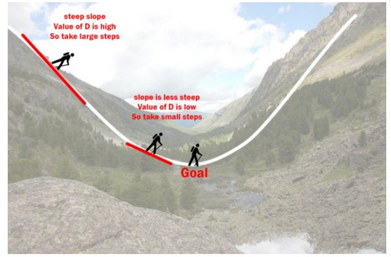


Gradient Descent

1. Initially let $m = 0$ and $c = 0$. Let L be our learning rate. This controls how much the value of m changes with each step. L could be a small value like 0.001 for good accuracy.
2. Calculate the partial derivative of the loss function with respect to m, and plug in the current values of x, y, m and c in it to obtain the derivative value $\boldsymbol{D}$.

$$D_m = \frac{1}{n}\sum_{i=0}^{n} 2(y_i - (mx_i + c))(-x_i)$$

$$D_m = \frac{-2}{n}\sum_{i=0}^{n} x_i(y_i - \bar{y}_i)$$



梯度下降



3. □m 是关于 □ 的偏导数值。类似地，我们来求关于 □、□$_c$ 的偏导数：

$$D_c = \frac{-2}{n}\sum_{i=0}^{n}(y_i - \bar{y}_i)$$

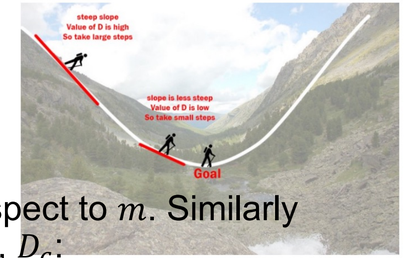
4. 现在我们使用以下方程更新 □ 和 □ 的当前值：

$$m = m - L \times D_m$$

$$c = c - L \times D_c$$

5. 我们重复这一过程，直到损失函数的值非常小，或者理想情况下为 0（这意味着 0 误差或 100% 准确率）。此时剩下的 □ 和 □ 的值将是最优值。

Gradient Descent



3. D_m is the value of the partial derivative with respect to m. Similarly let's find the partial derivative with respect to c, D_c:

$$D_c = \frac{-2}{n}\sum_{i=0}^{n}(y_i - \bar{y}_i)$$

4. Now we update the current value of m and c using the following equation:

$$m = m - L \times D_m$$

$$c = c - L \times D_c$$

5. We repeat this process until our loss function is a very small value or ideally 0 (which means 0 error or 100% accuracy). The value of m and c that we are left with now will be the optimum values.

迭代梯度下降

- 执行前向传播以计算损失
- 反向运行以计算梯度
- 对每个参数向量求损失和梯度的平均值
- 在每次迭代中，沿与梯度相反的方向更新每个参数向量，以使损失降低

Iterative Gradient Descent

- Run forward to compute the loss
- Run backward to compute the gradients
- Average the loss and the gradient for each parameter vector
- At each iteration, update each parameter vector in the direction opposite to its gradient so the loss decreases

梯度下降

更通用的解决方案?

自动微分!



Gradient descent

A more general solution?

Autograd!



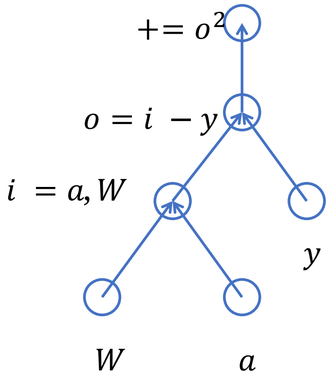
计算图

前向
- 分解为基本操作
- 构建有向无环图来表示计算

反向
- 通过链式法则求梯度

$$\frac{=y}{=x} = \frac{=y}{=\partial_n}\frac{=\partial_n}{=\partial_{n-1}}0\frac{=\partial_2}{=\partial_1}\frac{=\partial_1}{=x}$$

假设 $z = \mathbf{x}, \mathbf{w} - y\,2$



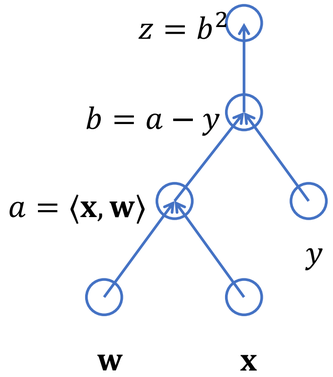
Computation Graph

Forward
- Decompose into primitive operations
- Build a directed acyclic graph to present the computation

Backward
- Get the gradient by chain rule

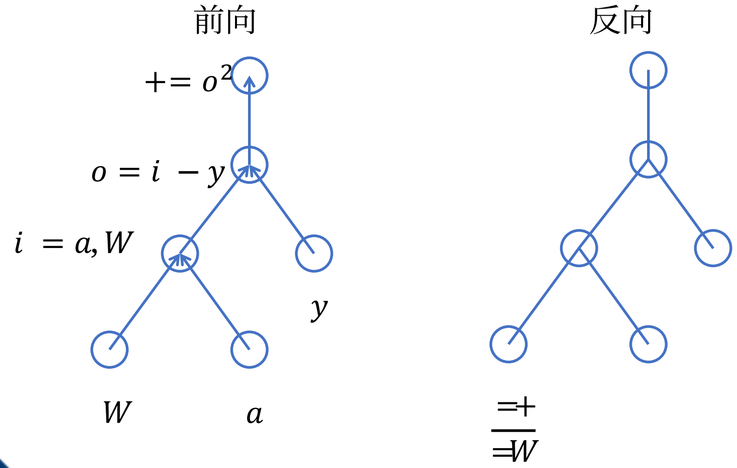
$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u_n}\frac{\partial u_n}{\partial u_{n-1}}\cdots\frac{\partial u_2}{\partial u_1}\frac{\partial u_1}{\partial x}$$

Assume $z = (\langle \mathbf{x}, \mathbf{w}\rangle - y)^2$



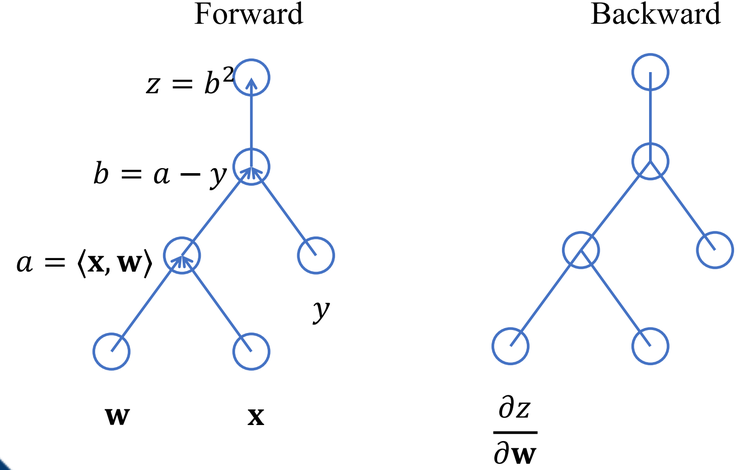
反向累积

$z = \langle \mathbf{x}, \mathbf{w} \rangle - y_2$ 假设



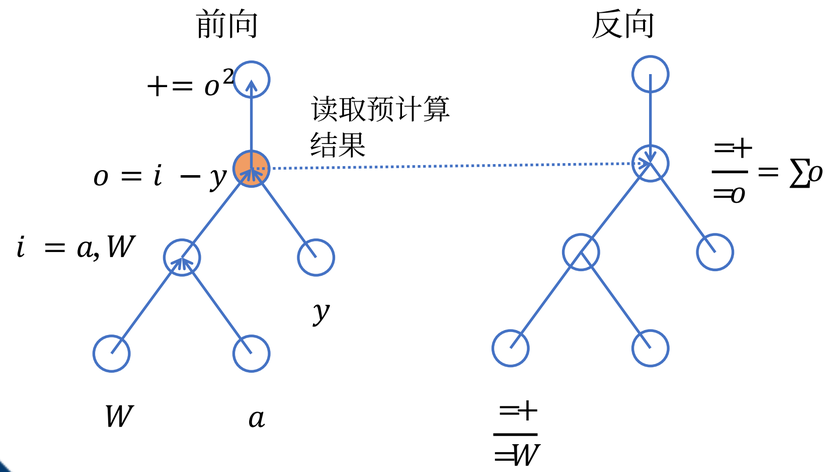
Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



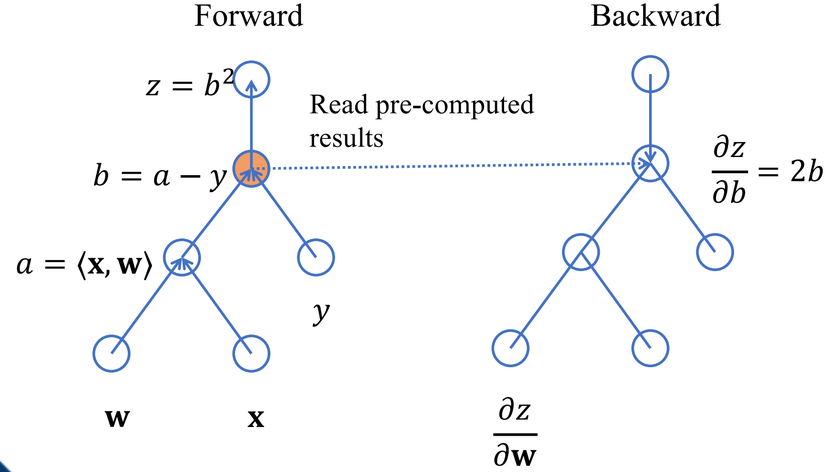
反向累积

$z = \langle \mathbf{x}, \mathbf{w} \rangle - y_2$ 假设



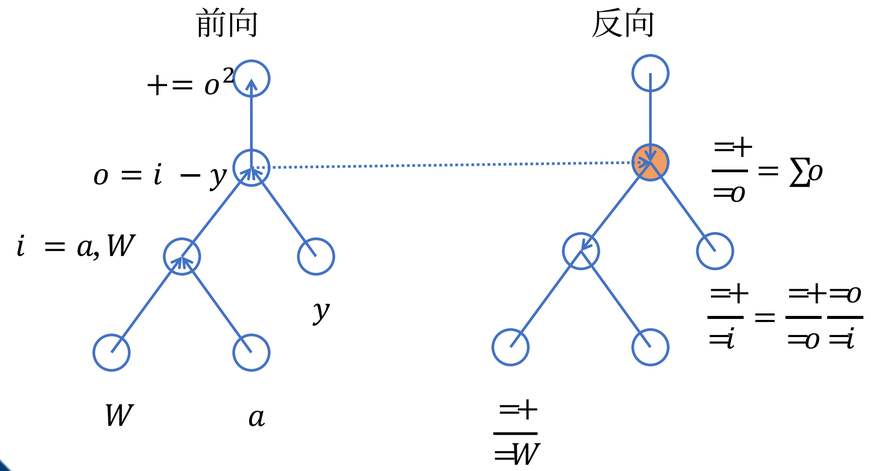
Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



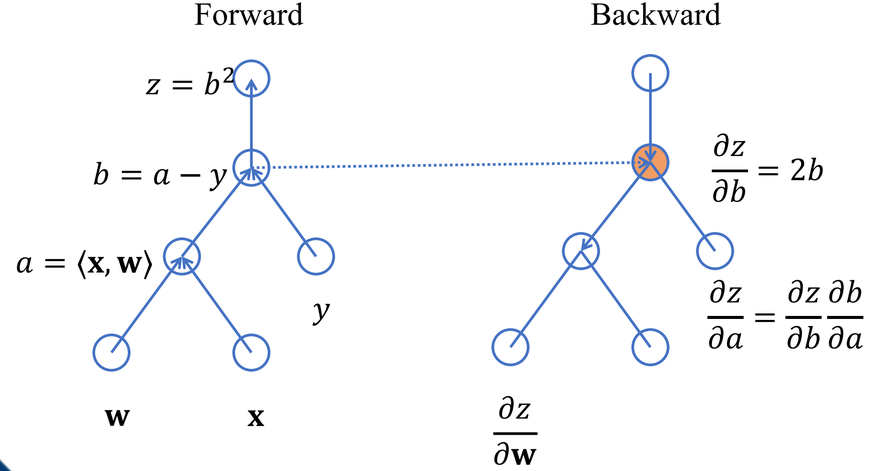
反向累积

$z = \langle \mathbf{x}, \mathbf{w} \rangle - y_2$ 假设



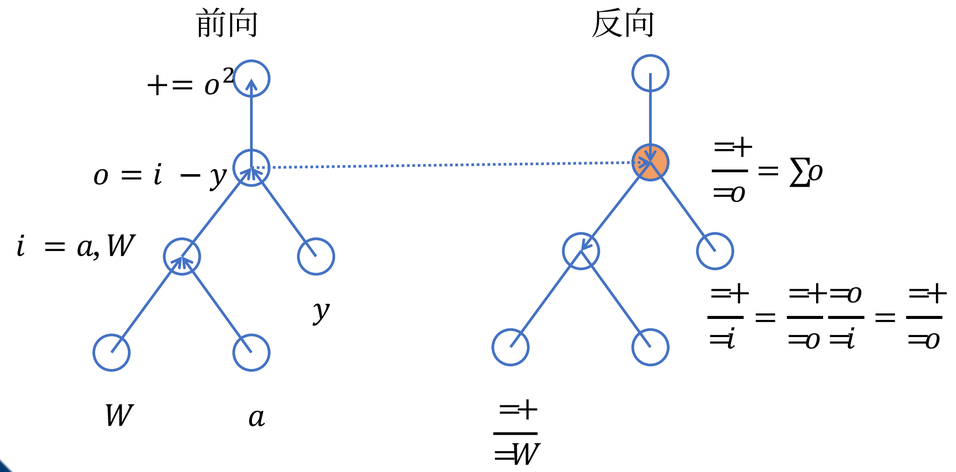
Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



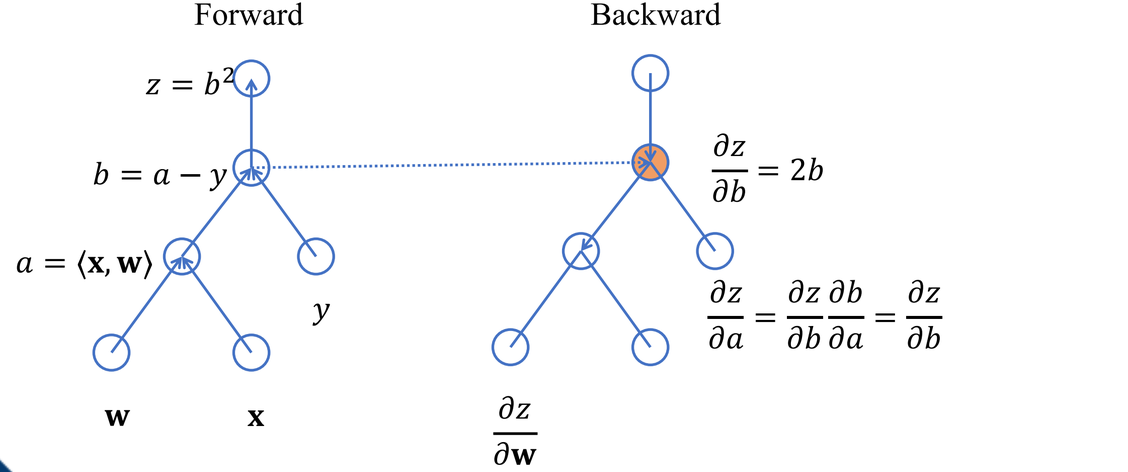
反向累积

$z = \langle \mathbf{x}, \mathbf{w} \rangle - y_2$ 假设



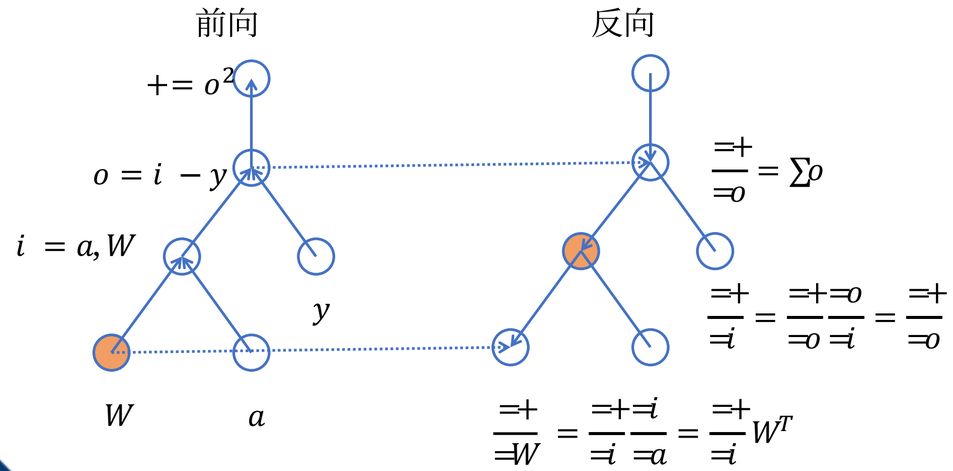
Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



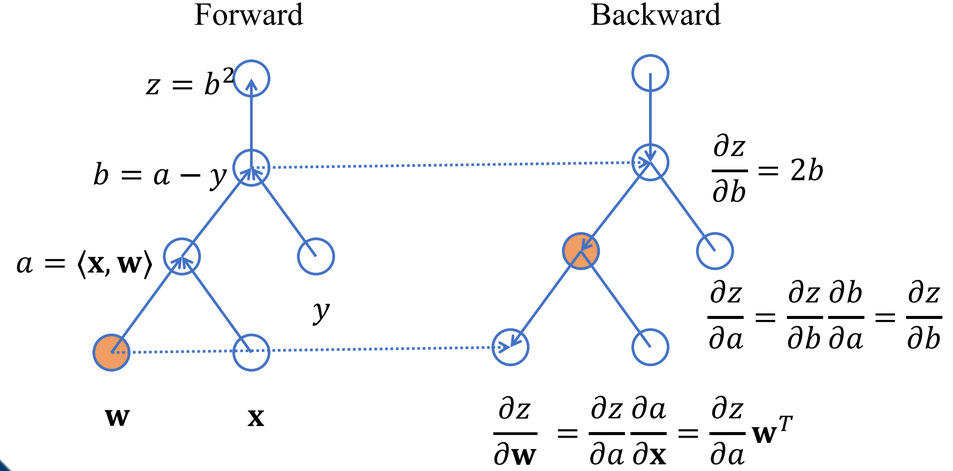
反向累积

$z = \langle \mathbf{x}, \mathbf{w} \rangle - y\ _2$ 假设



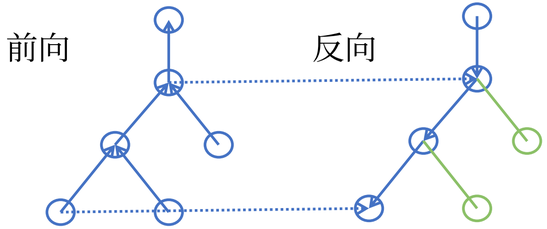
Reverse Accumulation

Assume $z = (\langle \mathbf{x}, \mathbf{w} \rangle - y)^2$



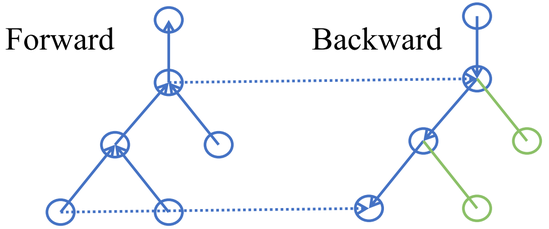
反向累积总结

- 构建计算图
- 前向：求值计算图，存储中间结果
- 反向：按相反顺序求值计算图
  - 消除不需要的路径



Reverse Accumulation Summary

- Build a computation graph
- Forward: Evaluate the graph, store intermediate results
- Backward: Evaluate the graph in a reversed order
  - Eliminate paths not needed



随机梯度下降

$$W \leftarrow W - \lambda \frac{\partial L}{\partial W} = W - \lambda \frac{\partial \frac{1}{N} \sum_{i=1}^{N} L_i}{\partial W}$$

当 N 很大时计算代价高昂

使用一批样本进行近似。B 可以根据内存/学习速度来选择.....

$$W \leftarrow W - \lambda \frac{\partial L}{\partial W} = W - \lambda \frac{\partial \frac{1}{B} \sum_{i=1}^{B} L_i}{\partial W}$$

Stochastic Gradient descent

$$W \leftarrow W - \lambda \frac{\partial L}{\partial W} = W - \lambda \frac{\partial \frac{1}{N} \sum_{i=1}^{N} L_i}{\partial W}$$

Expensive when N is large

Use a batch of samples for approximation.
B can be chosen based on the memory/learning speed…

$$W \leftarrow W - \lambda \frac{\partial L}{\partial W} = W - \lambda \frac{\partial \frac{1}{B} \sum_{i=1}^{B} L_i}{\partial W}$$